

Sandboxing untrusted components. When a component’s code cannot be trusted, language-level access control is insufficient, since a malicious component with access to the host runtime can reach the underlying objects directly, rendering such checks moot. Sandboxing requires an execution boundary beyond the reach of language-level means, such as software fault isolation [59], a separate language runtime, a sandboxed process, or a virtualized container [60]. Whatever the mechanism, the untrusted component runs in its own sandboxed context and reaches host-provided dependencies through a bridge, generalizing the cross-process invocation of Section 6.2: the same transparency argument renders this bridged access indistinguishable from local injection. On the host side, the bridge is an ordinary fiber whose capabilities can be attenuated by the access control described above.

6.4. Language Independence and Selection

Although Cordis is implemented in TypeScript, the context paradigm is language-agnostic: spatiotemporal composability is defined only by its two composability dimensions, and thus can be realized in any language that meets certain requirements along both. We analyze these requirements along each dimension in turn.

Temporal composability. At its most basic, temporal composability requires *closures*: a revertible effect pairs an action with an inverse, and that inverse must be captured as a value, along with the state it restores, so it can be replayed on teardown. Beyond this, a component’s code and the side effects of loading it must be introducible and retractable at runtime.

How a language meets this second requirement depends on its execution model. In managed runtimes, this takes the form of a programmatic module registry, where a loaded module can be evicted from the registry and garbage-collected once unreferenced; Node.js, for instance, exposes such a registry.⁶ Native code exposes no module registry, so introduction and retraction take the form of explicit dynamic linking and unlinking (e.g., `dlopen/dlclose` on Unix, `LoadLibrary/FreeLibrary` on Windows) [61], i.e., loading object code into a running process and later detaching it. WebAssembly takes one path or the other depending on its embedder: a module instance is reclaimed by the host’s collector under a managed embedder (e.g., a JavaScript host), or released when a native embedder drops it (e.g., Wasmtime). Across these mechanisms, the revertible effects model treats loading as an effect on the context, with inverses that undo the registration of symbols, types, or handlers the module introduced.

Spatial composability. Spatial composability requires a mechanism for components to declare their dependencies and for the runtime to provide and inject these dependencies. This reduces to a dependency injection (DI) problem [39], which manifests at two levels that differ across languages: how dependencies are *typed* and how their access is *mediated*.

At the type level, the language should provide a way for developers to express well-typed dependency access. A consumer obtains a coeffect by reading its key from the context, so the context type (Section 3.2.1) must record each key’s coeffect. Typeclasses (Haskell) [62] and traits (Rust) [63] achieve this by letting a provider extend the context type from its own module through an instance or `impl` [64]. TypeScript’s module augmentation [65] likewise lets a provider module merge declarations into the context type.

At the runtime level, dependency access must be dynamically mediated: the coeffect behind a key may change as providers are loaded and unloaded, and may be resolved differently across

⁶CommonJS exposes the module cache via `require.cache`; ES modules provide no public eviction API, though modules can still be managed through engine-internal interfaces.

contexts. The language therefore needs a way to interpose on access transparently, leaving the consumer’s code unchanged, e.g., via JavaScript’s Proxy object [66] or Python’s descriptor protocol (`__get__`) [67]. Absent such a primitive, runtime reflection [68, 69] can mediate access dynamically, at the cost of type safety and developer experience.

Across both levels, metaprogramming facilities supply the typing and the mediation together. Annotations [70] and decorators attach metadata to a declaration, which a processor expands into the accessor that mediates access; compile-time metaprogramming (e.g., Rust procedural macros, Scala macros [71], Zig `comptime`) emits, for each dependency, a typed declaration together with such an accessor, dispensing with a general-purpose interception primitive.

6.5. Mutual Dependencies and Component Granularity

In the reactive coeffect model, a dependency cycle simply leaves the involved components permanently inactive: given two components A and B , if A requires a key provided by B and B a key provided by A , neither’s satisfaction predicate can ever become true. Unlike deadlock in concurrent systems, which depends on the schedule and must be detected as it happens, this condition is predictable from the dependency declarations alone, so a runtime can report it when components are loaded.

In practice, most apparently mutual dependencies can be decomposed into finer-grained components that eliminate the cycle. Consider two components: a server (providing a network interface) and an access controller (enforcing authorization policies). The two components interact bidirectionally: the access controller mediates requests arriving at the server, and the server exposes an endpoint for modifying access-control policies. A monolithic design would make each component depend on the other. However, the two interaction directions are logically independent concerns. Decomposing them yields four components: server-core, access-control-core, request-mediation (depending on both cores to apply access control to incoming requests), and policy-management (depending on both cores to expose policy modification via the server). Through this approach, the cycle is eliminated because neither core depends on the other; only the integration components depend on both.

This decomposition is always possible in principle, since every bidirectional interaction can be factored into independent unidirectional bindings, but it increases the number of components: in the general case, given n mutually interacting components, the number of integration components can grow quadratically with n , since each pair of interacting components may require a distinct component for each direction of interaction. This does not affect correctness or runtime performance (components are lightweight), and finer granularity can be beneficial: users gain the ability to load only the specific integration bindings they need, effectively increasing the system’s composability. However, it may affect developer experience: more components require more configuration, more naming, and more cognitive overhead in understanding the dependency graph.

Mitigating this granularity cost is an engineering concern rather than a theoretical one. Practical strategies include package bundling (i.e., grouping related fine-grained components into a single installable unit), convention-based wiring (i.e., automatically connecting components whose names or types match a pattern), and scaffold tooling (i.e., generating boilerplate integration components from declarative specifications). These strategies preserve the formal guarantees of the acyclic model while reducing the authoring burden to something closer to the monolithic case.

6.6. Dependency Typing and Versioning

In the formal model, a dependency link is established purely by key identity: a component providing key k satisfies any component declaring k in its dependency set. The type family $\mathcal{V}_k$ ensures type-level agreement within a single compilation unit, but this guarantee breaks down when components are developed and built independently, which is a common scenario in component ecosystems. This breakage leads to two distinct problems.

Interface drift. A provider may modify the interface associated with k (adding fields, changing method signatures, altering behavioral contracts) between versions, while a consumer compiled against an earlier interface continues to declare the same key k . The dependency is satisfied at the coefficient level ($k \in \text{dom}(\sigma)$), yet the runtime value no longer conforms to the consumer’s expectations, leading to type errors, method-not-found failures, or silent behavioral divergence [72].

Key collision. Two independently developed providers may use the same key name k to denote entirely unrelated interfaces. Since key identity alone establishes the link, a consumer expecting one provider’s interface will accept the other’s value without any compatibility check. Unlike interface drift, where the provider and consumer at least share a common lineage, key collision involves no relationship whatsoever between the expected and actual types, making the resulting failures unpredictable and difficult to diagnose.

Both problems point to the same gap: the coefficient model provides only *nominal* linking (by key name) but no *versioned* or *structural* linking (by interface compatibility) [73]. We discuss three approaches to the gap, from most infrastructure-coupled to most language-agnostic.

Key namespacing. Extending the key space from K to $K \times P$, where P identifies the interface-defining package, eliminates key collision by construction: independently developed interfaces with the same local name occupy distinct keys. This is the most direct solution but also the most coupled: it embeds the package namespace into the formal model itself, making the system dependent on an external package registry for key identity.

Peer dependencies. A lighter coupling is to declare version constraints through the host-language package manager [74]. This is the approach Cordis currently adopts. Component dependencies are semantically *peer dependencies*: a component does not bundle its dependencies internally but expects the runtime context to supply them. Package managers with peer dependency support (e.g., npm) can enforce version compatibility: if the version of the package providing a key falls outside a consumer’s declared peer range, the incompatibility is caught at install time rather than surfacing as a runtime failure. However, this approach has two limitations: (1) it depends on providers faithfully adhering to semantic versioning, which is an unenforceable convention; (2) package managers typically resolve each dependency to a single version, which prevents loading components from multiple versions of the same package within one application.

Structural compatibility. A fully language-agnostic approach would replace the membership check $k \in \text{dom}(\sigma)$ with a compatibility predicate that verifies the provider’s actual interface structurally subsumes the consumer’s expectation. This is analogous to structural subtyping [75]: a provider satisfies a consumer if the provided interface is a subtype of the required interface. The challenge lies in defining this predicate language-agnostically: structural compatibility is straightforward for record types (with subtyping) but becomes complex for behavioral contracts (e.g., pre/postconditions [76], effect specifications [22]), and undecidable once parametric polymorphism introduces bounded quantification [77].

These three approaches address different aspects of the problem. Designing a unified dependency model that combines these approaches while preserving the dynamic composition guarantees of the coeffect model remains an open problem.

6.7. Co-Design with Languages and Operating Systems

Section 6.4 identifies the minimum a host language must supply for the context paradigm. This section takes up the converse question, what a language or operating system co-designed with the paradigm can offer beyond that minimum.

Co-design with languages. A language designed around the context paradigm can improve on a library in two respects: the semantics it gives to contexts, and the primitives it gives to effects and coeffects.

Such a language can make the context implicit again while preserving the context semantics of Section 3.3. An imperative language already runs every statement against an implicit context, and that single context neither tracks effects nor resolves coeffects. The context paradigm instead distinguishes multiple contexts, where an operation either modifies the context it runs against or derives another from it (Definition 23). An in-place realization modifies the ambient context, just as an imperative language does. A derived realization instead introduces a separate context, for which the language must provide a construct. Making the context implicit brings both an ergonomic and a safety benefit. (1) In a library realization, every function involving effects or coeffects takes the context as an ordinary argument or a receiver, as in Section 5.1. Where the language supplies the context implicitly, functions no longer need to take it. (2) Every context carries its own lifecycle state and committed view (Section 4.1). A library realization passes a context as an ordinary variable, so a component may reach another component's context by mistake, through a closure or a global variable. An effect it installs there then leaks out of its own lifecycle, and a coeffect it reads there escapes its dependency specification. Making the context implicit closes both.

Such a language can also make effects and coeffects known to its compiler. (1) For effects, an effect iterator (Definition 17) allocates a closure at every step to hold the inverse together with the state it restores. With syntax for performing an effect, a compiler can emit a single state machine for the whole iteration and hold those inverses in its frame. (2) For coeffects, the coeffect specification can be admitted into the type system, with two benefits. First, a dependency cycle is reported at compile time instead of being left to the runtime (Section 6.5). Second, a dependency can be compared by the structure of its type rather than by key identity alone, as row types do [28], which is type-level support for the structural compatibility of Section 6.6.

Co-design with operating systems. Section 1.2.3 observes a coarse-grained substitute for dynamic composability, where the operating system supplies temporal composability at the granularity of a process, and the container orchestrator above it supplies spatial composability at the granularity of a service. An operating system co-designed with the paradigm would support fine-grained composition, by making the coeffect specification a component declares the whole of what it can reach, and by providing its own resources as coeffects.

Such an operating system can supply the sandbox that Section 6.3 defers to a mechanism outside the language. It does so by bounding a component to the dependencies it declares, supplying them when the component is loaded and leaving nothing else reachable from within it, as a WebAssembly module receives its imports from its embedder at instantiation [78]. It

can also provide the coeffect isolation and interception of Section 3.2.3 as abilities of its own, binding a key differently for each component and mediating the accesses it supplies.

Such an operating system can also provide its own resources as coeffects. A resource lying outside the boundary is made revertible where the runtime records each acquisition against the component that made it (Section 6.1), and every runtime keeps a record of its own. An operating system that provides the resource as a coeffect keeps that record once, since it is the party that hands the resource out and can attribute it to the component that asked. Memory and file descriptors are the immediate candidates, and tracking them for the sake of recovery has been done at the kernel interface [79, 80]. Furthermore, an operating system can make revertible some of the operations Section 6.1 can only withhold or compensate for. A system that performs a write to persistent storage transactionally can roll it back [81], and one built on copy-on-write or immutable storage reaches an earlier state by moving a pointer [82, 83].

7. Related Work

Dynamic composability intersects several established research areas. We survey the most relevant lines of work and distinguish our contribution from each of them.

7.1. Effect and Coeffect Systems

Section 2 reviewed effects and coeffects as the theoretical pillars underlying our work. We first situate the monadic effect systems now common in industrial practice, then survey three research lines that extend effects and coeffects in directions relevant to Cordis: recasting algebraic effects as capabilities, giving effects a reversible semantics, and unifying effects and coeffects under a single graded discipline.

Monadic effect systems. One family of libraries encodes effects in the type systems of existing general-purpose languages, representing them as monadic values that a runtime executes. ZIO in Scala [84] models a computation as $\text{ZIO}[R, E, A]$ and Effect-TS in TypeScript [85] as $\text{Effect}<A, E, R>$, a generic type whose parameters describe its result, its typed errors, and the services its context must supply; the fp-ts library [86] encodes the same error and requirement channels through Reader-based monad transformers. Two traits separate these systems from Cordis. First, the tracking costs a monadic embedding: a program obtains it only by being written inside the effect type, whereas Cordis tracks effects as an overlay over ordinary host code. Second, a requirement is discharged by interpretation, an installed service that supplies its operations, and when that service is withdrawn what its operations performed remains in place; Cordis instead pairs each effect with an inverse and re-resolves requirements as providers come and go (Section 3.1, Section 3.2).

Algebraic effects as capabilities. Algebraic effects (Section 2.1) make effect operations visible to the type system. The extension closest to our work is Brachthäuser et al.’s Effekt language, which reinterprets effect types as *capabilities* [87, 88]: an effect type expresses what a computation requires from its context rather than what side effects it may produce. This perspective, like ours, treats the context as a mediator of capabilities. Cordis and Effekt differ in two respects. (1) In purpose, algebraic effects make effects visible to enable *modular interpretation*, giving one operation many handler semantics, whereas Cordis makes them visible to enable *tracking and reversion*, pairing every context transformation with an inverse. (2) In setting, Effekt disciplines effects statically at the type level, defaulting to scope-based reasoning in which capabilities are

second-class and confined to their lexical scope, and recovering first-class use through boxing, which lifts that restriction by tracking captured capabilities in types; Cordis instead disciplines effects at runtime, aiming at complete resource recovery on component removal; Section 6.7 takes up what a language that made the context second class in this sense would offer.

Reversible effect semantics. A parallel line gives effects a reversible semantics rather than an interpretive one. Heunen et al. [89] model side effects in a reversible setting by adapting Hughes’ arrows to *dagger arrows* and *inverse arrows*, capturing effects such as serialization and mutable store whose operations admit inverses. This is the formal account closest to our revertible effects: both pair each effect with the means to undo it rather than discharging it through a handler. The two differ in where reversibility resides, and in how much of it they demand. Heunen et al. work in a denotational, categorical setting where reversibility is a global property, guaranteed by construction since every computation is invertible, and the inverse is two-sided and recovered from the categorical structure. Cordis tracks inverses at runtime and requires less of them: not that the whole computation be reversible, but that each atomic effect admit a one-sided inverse, supplied by the caller at the point of application rather than derived, from which the inverse of any composite follows by composition (Section 3.1).

Graded types as unified effects and coeffects. Orchard et al. [90] proposed *graded modal types* as an umbrella notion encompassing both effect reasoning (via graded monads) and co-effect reasoning (via graded comonads), realized in the Granule language, demonstrating that a single type system can track both what a computation does and what it needs; more recent work extends coeffects to imperative Java-like languages [91, 92] and to call-by-push-value [93]. All of these operate at the type level: effects and coeffects are static annotations checked at compile time over lexically fixed scopes. Our contribution is orthogonal to this analysis: we lift the same two notions to runtime mechanisms, which lets Cordis handle dynamic composition. Temporal retraction and spatial dependency are re-resolved as the set of loaded components evolves, instead of being settled once over a fixed program text.

7.2. Programming Paradigms

The context paradigm (Section 3.3) mediates every effect and coeffect through an explicit context. This section first compares it with the functional and imperative treatments of side effects, and then with two established paradigms, one sharing our terminology and the other our treatment of crosscutting concerns.

Explicit threading and implicit mutation. Purely functional languages make effects explicit in types: the State monad $S \rightarrow (A, S)$ [23] threads the environment through every computation, securing equational reasoning at the cost of the threading itself, every function on the call path accepting and returning the state whether or not it touches it; the monadic effect systems of Section 7.1 are this pole in industrial form. Imperative languages leave effects and dependencies implicit at the call site, so reading what a call does to the system means reading its implementation transitively, and moving or removing a call may silently break distant invariants. The context paradigm takes the traceability of the first treatment and the ergonomics of the second: effects and coeffects pass through a context the component holds, so each operation is attributable to the context it was invoked on and hence to the component, and everything else stays ordinary host code. The paradigm is in this sense an overlay, realizable atop a language of either style: it fixes each operation’s denotation and leaves its realization to the host language, in place where the host mutates and derived where it stays pure (Definition 23).

Context-oriented programming. COP [94, 95] equips a language with *layers*—partial method and class definitions that are activated and deactivated at runtime according to the execution context, so that behavior adapts without the base code naming its context dependencies [96]. COP and Cordis coincide in treating context as a first-class, runtime-mutable entity and in activating and deactivating behavior dynamically, but the resemblance is nominal. In COP, “context” denotes the ambient execution situation (e.g., location, user, mode), and activation changes method dispatch within a dynamically scoped extent; a layer neither tracks the side effects it induces nor reverts them, and activation is not governed by dependency satisfaction. In Cordis, the context is the Γ_∞ entity mediating effects and coeffects: activation runs a component’s revertible effects and is driven by reactive coeffect satisfaction (Section 3.2), and deactivation reverts them in full. COP varies what behavior runs; Cordis composes and reverts what effects and dependencies a component installs. Their difference is one of trade-off. COP folds activation into the host language’s method dispatch, gaining dynamically-scoped layer extents at the cost of language specificity, whereas Cordis, as a language-agnostic overlay, resolves activation reactively over a shared context. Cordis can thus express as a coeffect only COP’s global, value-driven fragment: context-dependent selection among implementations, but not dynamically-scoped activation.

Aspect-oriented programming. AOP [97, 98] modularizes a crosscutting concern into an *aspect*: a *pointcut* that quantifies over *join points* selected in the base program, and *advice* woven in at each. Cordis addresses the same problem of contextual behavior that would otherwise scatter across components, but its analogue of an aspect is a coeffect: a shared point of mediation many components declare a dependence on, so that crosscutting behavior can be reshaped there without editing any of them. The two paradigms then differ on two axes. (1) *Declaration versus obliviousness*: an AOP pointcut is oblivious and quantified, matching arbitrary join points whose code is unaware it is advised, whereas Cordis confines crosscutting to the coeffects each component declares, so its reach is exactly that declared surface. This yields determinacy and traceability: an application orchestrator can inspect and govern what cross-cuts a component at the configuration layer, without reading or analyzing its source, whereas an AOP concern is legible only through the aspects that quantify over it. (2) *Lifecycle integration*: a crosscutting change in Cordis is carried by a component’s effects, reverted when the component unloads and propagated reactively to its dependents, so it is one move within the dynamic composition model; dynamic-AOP systems [99, 100] can also weave and unweave at runtime, but as a standalone operation, neither bound to a component’s lifecycle nor triggering re-resolution among the advised code.

7.3. Temporal Composability

Temporal composability concerns replacing or removing a component in a running program while recovering the effects it installed. Prior approaches divide by how they treat a departing component’s state and effects: carrying state forward to a successor version, recovering effects through developer-authored cleanup, reversing effects automatically within a scope fixed in advance, or reclaiming resources from a record the runtime accumulates by interposing on an interface.

Stateful forward migration. A broad family of systems replaces components in a running program without downtime by carrying their state forward across versions. All observe the same timing discipline: a component may be swapped only once it reaches a safe, interaction-free point. Kramer and Magee established this criterion as *quiescence* [53], which Vandewoude et al. later relaxed to the less disruptive *tranquility* [54]; our rolling-update pattern (Section 6.2)

enforces it by draining in-flight requests before unloading a provider. Dynamic software updating (DSU) then migrates state forward through hand-written transformation functions: Hicks et al.’s general-purpose DSU for C [101], Stoyke et al.’s type-safe update points via con-freeness analysis [102], and Hayden et al.’s Kitsune [103] all map old-version data to new-version representations, inheriting heap objects, open files, and connections in place while re-initializing whatever is left unmigrated. The same discipline extends to persistent state: Overeem et al. [104] convert a running event store’s data between schema versions through hand-written upgrade operations while keeping the system available. Erlang/OTP [15] takes the same stance at the process level, migrating state through `code_change/3` and recovering from faults by restarting supervised processes rather than reverting their effects; JavaScript’s Hot Module Replacement (e.g., webpack [48], Vite [49]) does the same at the module level, handing state forward through the `module.hot` or `import.meta.hot` API across a reload. Compared with Cordis’s module replacement (Section 5.2), these approaches migrate in-memory state more gracefully: Cordis reverts the old component’s tracked effects and reapplies the new component’s from a clean slate, so a component’s own in-memory state does not survive a reload unless placed in a longer-lived dependency, and layering DSU-style forward migration atop revertible effects is future work. Cordis’s approach is nonetheless more general in two respects: it needs no hand-written migration functions of the kind DSU and HMR require, and it supports unloading a component entirely and recovering its resources, not merely updating one in place.

Developer-authored recovery. A second family recovers a component’s effects through cleanup or compensation logic that the developer writes by hand. Plugin lifecycle conventions (e.g., OSGi [52], Eclipse’s extension points, IntelliJ and VSCode) delegate cleanup to developer-written unload callbacks; the Command pattern [105] encapsulates an operation together with an undo method for undo/redo stacks; the saga model [51] structures a long-lived transaction as steps each paired with a compensating action; algebraic effect handlers can attach finalizers that run on teardown [106]; and event sourcing [107] retracts state by appending compensating events rather than executing an inverse at all. In all of them the inverse is an unenforced duty, decoupled from the operation, so that a forgotten one leaks resources silently (as documented empirically in Section 1.2.1). React’s `useEffect` hook [108] comes closest to pairing an effect with its inverse structurally, returning a cleanup the runtime invokes before each re-execution and on unmount. Its shortfall is composability: a hook may be called only at the top level of a component or another hook, never inside a conditional, loop, or nested function, and its effect body accepts neither an async function nor an iterator. Effects thus cannot be assembled from other effects or interleaved with control flow, leaving nothing from which a composite inverse could be derived. Cordis effects carry no such restriction: they are ordinary operations that compose freely and may run asynchronously, and require a hand-written inverse only for each atomic effect, from which the inverse of any composite is derived by composition, so that assembling existing effects requires writing no inverses at all. This structural pairing of every effect with its inverse makes complete recovery an invariant of the system rather than a matter of developer discipline.

Statically scoped reversal. A third family reverses effects automatically, by construction, but confines reversal to a scope fixed in advance. Software transactional memory [109, 110], descended from hardware transactional memory [111], records a read/write log so that a group of memory operations either commits or aborts, rolling memory back to its pre-transaction state. Reversible computing, from Landauer and Bennett’s thermodynamic analyses [112, 113] to reversible languages such as Janus [114], goes further and makes every step of a whole computation globally invertible. Reversible process calculi build backtracking into the

semantics itself: RCCS [115] carries a memory alongside each process and admits a step to be taken back when the past it leads to is causally equivalent, and Phillips and Ulidowski [116] derive reversible operators for CCS, ACP, and CSP uniformly while preserving their forward operational semantics. Their causal-consistency criterion is the concurrent counterpart of the order Cordis’s recovery follows, an accumulator applying a component’s own inverses in last-in-first-out order and the guard of Section 4.2.2 deferring a provider’s withdrawal until its consumers have deactivated (Theorem 70). The reach, however, is fixed by the semantics, every action performed remaining undoable, whereas a Cordis component supplies an inverse for each atomic effect and its accumulator brings the context back to where its composition began. Linear types [117], RAI [4], and Rust’s ownership system [63] tie a resource’s release to a lexical region. Each fixes the scope and reach of reversal statically; Cordis, by contrast, fixes no such scope in advance: it reverts arbitrary context operations over a component’s lifecycle, and treats lexical resource management as complementary, appropriate for local resources within a single component. Verification supplies the same pairing at the granularity of a data structure. Kim and Rinard specify and verify an inverse for every state-changing operation of a collection of set and map implementations, together with the conditions under which two operations commute, reasoning on abstract state so that orders leaving equivalent rather than identical structures count as commuting [118]. Both of the ingredients Section 3.4 requires of a coeffect are therefore mechanically checkable at the interfaces most keys publish, and their reason for preferring an inverse to a saved copy of the state is the one Section 3.1 acts on. Their inverses do not compose, each being verified for one operation, whereas a Cordis accumulator carries the inverses of a whole lifecycle.

Interposed reclamation. A fourth family reclaims what a component acquired without the component itself supplying the inverses, by recording its acquisitions at an interface the runtime controls. Nooks [79] wraps every call crossing the boundary between the Linux kernel and its loadable extensions, so that the kernel objects an extension touches pass through an object tracker whose record tells the recovery manager what to release when the extension fails; shadow drivers [80] tap the same calls from the other side, recording the requests and configuration that determine a driver’s state so that a restarted instance can be restored to it. Akeso [119] obtains the record by compiler instrumentation instead, dividing kernel execution into nestable recovery domains that log their state changes and cross-thread dependencies, and rolling a faulting request back together with every domain that depends on it. Reclamation thus follows from a record the runtime maintains rather than from cleanup the developer remembers to write, which makes this family the closest systems-level precedent for revertible effects. It differs from Cordis in vocabulary and in reach. The platform fixes what can be recorded, whether as release code per kernel object type, one shadow per driver class, or an inverse per instrumented allocator, so a component may hold only resources the platform already knows how to release; a Cordis component instead introduces effects of its own and supplies an inverse for each atomic one (Section 3.1). Reclamation is likewise bounded by a request that commits or a restart of the same extension, whereas Cordis reverts over a component’s whole lifetime and propagates removal to its dependents, which release their own effects in turn (Section 3.2).

7.4. Spatial Composability

Spatial composability concerns how a component’s dependencies on others are declared and bound. Prior mechanisms divide by how binding responds to change: wiring dependencies once at initialization, reacting to the availability of whole components, or propagating change at the granularity of individual values.

Initialization-time dependency wiring. Two established mechanisms wire components together at initialization time. Dependency injection frameworks [39] (e.g., Spring [120], Guice, Angular, Inversify) inject dependencies into components at initialization, and UI framework context (e.g., Vue.js’s `provide/inject` and React’s Context API) passes them along a component tree. Some support dynamic scoping (e.g., Spring’s prototype/request scopes, Angular’s hierarchical injectors), but neither re-resolves reactively: when a provider is replaced or removed at runtime, existing dependents are neither deactivated nor re-initialized, and none offers lifecycle management of the kind our component state machine provides. Cordis’s reactive coeffects (Section 3.2) supply this: the notification mechanism triggers lifecycle transitions whenever the satisfaction predicate changes.

Availability-reactive component models. The closest precedent to our reactive coeffects reacts to service availability. OSGi’s Declarative Services and iPOJO [121, 122] let components declare provided and required services, with the runtime automatically activating and deactivating them as services appear and disappear; iPOJO’s Gravity project [122] explicitly targets autonomous runtime adaptation to changing service availability, and its `provide/require` model directly prefigures Cordis’s `ctx.provide/ctx.get` pattern. R-OSGi [55] extends the same abstraction transparently to distributed settings via RPC, mapping network failures to service-withdrawal events, a pattern Section 6.2 discusses as an extension of the Cordis model. All these systems recover through a deactivation callback, which is limited in two ways. First, the callback is hand-written, so resource safety rests on developer discipline and a forgotten one leaks silently. Second, the callback is synchronous: should teardown require an asynchronous exchange with the departing dependency, the frameworks offer no protocol to await it, forcing a blocking wait against a reference that may already be stale. Cordis’s reactive coeffects close both gaps: deactivation reverts the dependents’ accumulated effects, and its inertial Unloading state (Section 4.4) runs asynchronous teardown to completion before acting on further change.

Value-level reactivity. Functional reactive programming (FRP) [123] and its modern incarnations (e.g., signals [124, 125] in SolidJS, Vue’s reactivity system, Angular Signals) propagate change at a *value-level* granularity: when a signal changes, derived computations are re-evaluated synchronously or under a scheduler [126]. Cordis’s reactive coeffects act at a *component-level* granularity, adding asynchronous lifecycle semantics that value-level propagation does not model. The same granularity difference runs the other way for consistency: propagating in a turn, in an order the dependency graph fixes, lets FRP require that no derived computation read a mixture of updated and stale inputs, which is *glitch freedom* [127], whereas Cordis has no counterpart of a turn, orchestration actions arriving one at a time, and guarantees only that no single transition straddles two resolutions of its coeffects (Theorem 71). The two are complementary rather than competing: a Cordis coeffect can itself carry reactive values, and a component updates on only the parts it actually consumes, refining component-level reactivity into finer-grained reactive coeffects that span both levels.

8. Conclusion

We have presented a formal foundation for dynamic composability by lifting the classical concepts of effects and coeffects to runtime mechanisms. *Reversible effects* address local temporal composability: every context transformation carries an inverse that the runtime holds, and both tracking and recovery preserve composition, so the context is recovered upon component removal. *Reactive coeffects* address local spatial composability: every context change is classified against a component’s coeffect specification as activating, deactivating, or neutral, and the

classification drives its activation and deactivation. We then unify the effect context and the coeffect context into a single context type and mediate every effect and coeffect through it, yielding a discipline we call the *context paradigm*; the mediation induces an observational equivalence up to which the effects of distinct components attain independence. Combining these mechanisms into the notion of a *component*, we give a calculus of dynamic composition whose metatheory carries spatiotemporal composability from a single component to a whole system of interleaved components. We realize this paradigm as the *Cordis* meta-framework, with a core library providing effect tracking and coeffect resolution, as well as a declarative component loader with configuration reconciliation and hot module replacement. The Koishi case study validates the design of *Cordis* in a production system with over 4000 community plugins.

Beyond human-curated plugin ecosystems, a compelling direction for future validation is self-evolving agent harnesses (Section 1.2.2), where an AI agent generates and replaces its own harness components continuously and with little human oversight. Applying *Cordis* in such a setting would validate the temporal guarantees of complete recovery under rapid component replacement, as well as the spatial guarantees of dependency coordination under frequent topological change. Such validation would demonstrate the paradigm’s applicability as a foundation for recoverable, coordinated, and continuous self-evolution in agent harnesses and other autonomous systems.

References

- [1] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972, doi: 10.1145/361598.361623.
- [2] D. Birsan, “On Plug-ins and Extensible Architectures,” *ACM Queue*, vol. 3, no. 2, pp. 40–46, 2005, doi: 10.1145/1053331.1053345.
- [3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.
- [4] B. Stroustrup, *The Design and Evolution of C++*. Addison-Wesley, 1994.
- [5] S. Marlow, S. Peyton Jones, A. Moran, and J. Reppy, “Asynchronous Exceptions in Haskell,” in *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation*, in PLDI ’01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 274–285. doi: 10.1145/378795.378858.
- [6] L. Cardelli, “Program Fragments, Linking, and Modularization,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 1997)*, ACM Press, 1997, pp. 266–277. doi: 10.1145/263699.263735.
- [7] C. Szyperski, *Component Software: Beyond Object-Oriented Programming*, 2nd ed. Addison-Wesley, 2002.
- [8] R. Lopopolo, “Harness Engineering: Leveraging Codex in an Agent-First World.” [Online]. Available: <https://openai.com/index/harness-engineering/>
- [9] Anthropic, “Harness Design for Long-Running Application Development.” [Online]. Available: <https://www.anthropic.com/engineering/harness-design-long-running-apps>

- [10] L. Wang *et al.*, “A Survey on Large Language Model Based Autonomous Agents,” *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024, doi: 10.1007/s11704-024-40231-1.
- [11] Y. Qin *et al.*, “Tool Learning with Foundation Models,” *ACM Computing Surveys*, 2025, doi: 10.1145/3704435.
- [12] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” *CoRR*, vol. abs/2310.08560, 2023.
- [13] T. Guo *et al.*, “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, in IJCAI 2024. 2024, pp. 8048–8057. doi: 10.24963/ijcai.2024/890.
- [14] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large Language Models as Tool Makers,” in *Proceedings of the Twelfth International Conference on Learning Representations*, in ICLR 2024. 2024. [Online]. Available: <https://openreview.net/forum?id=qV83K9d5WB>
- [15] J. Armstrong, “Making Reliable Distributed Systems in the Presence of Software Errors,” Doctoral dissertation, 2003. [Online]. Available: https://erlang.org/download/armstrong_thesis_2003.pdf
- [16] E. Moggi, “Notions of computation and monads,” *Information and Computation*, vol. 93, no. 1, pp. 55–92, 1991, doi: 10.1016/0890-5401(91)90052-4.
- [17] G. Plotkin and J. Power, “Adequacy for Algebraic Effects,” in *Foundations of Software Science and Computation Structures*, F. Honsell and M. Miculan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–24.
- [18] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: unified static analysis of context-dependence,” in *Proceedings of the 40th International Conference on Automata, Languages, and Programming - Volume Part II*, in ICALP’13. Riga, Latvia: Springer-Verlag, 2013, pp. 385–397. doi: 10.1007/978-3-642-39212-2_35.
- [19] M. Gaboardi, S.-ya Katsumata, D. Orchard, F. Breuvar, and T. Uustalu, “Combining effects and coeffects via grading,” in *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, in ICFP 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 476–489. doi: 10.1145/2951913.2951939.
- [20] A. Church, “A Formulation of the Simple Theory of Types,” *The Journal of Symbolic Logic*, vol. 5, no. 2, pp. 56–68, 1940, doi: 10.2307/2266170.
- [21] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [22] J. M. Lucassen and D. K. Gifford, “Polymorphic Effect Systems,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL ’88. San Diego, California, USA: Association for Computing Machinery, 1988, pp. 47–57. doi: 10.1145/73560.73564.
- [23] P. Wadler, “Monads for functional programming,” in *Program Design Calculi*, M. Broy, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 233–264.
- [24] G. Plotkin and J. Power, “Notions of Computation Determine Monads,” in *Foundations of Software Science and Computation Structures*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 342–356. doi: 10.1007/3-540-45931-6_24.

- [25] G. Plotkin and M. Pretnar, “Handlers of Algebraic Effects,” in *Programming Languages and Systems (ESOP)*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7.
- [26] M. Pretnar, “An Introduction to Algebraic Effects and Handlers. Invited tutorial paper,” *Electron. Notes Theor. Comput. Sci.*, vol. 319, no. C, pp. 19–35, Dec. 2015, doi: 10.1016/j.entcs.2015.12.003.
- [27] D. Leijen, “Koka: Programming with Row Polymorphic Effect Types,” *Electronic Proceedings in Theoretical Computer Science*, vol. 153, pp. 100–126, Jun. 2014, doi: 10.4204/eptcs.153.8.
- [28] D. Leijen, “Type directed compilation of row-typed algebraic effects,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, in POPL '17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: 10.1145/3009837.3009872.
- [29] A. Bauer and M. Pretnar, “Programming with algebraic effects and handlers,” *Journal of Logical and Algebraic Methods in Programming*, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: 10.1016/j.jlamp.2014.02.001.
- [30] K. Sivaramakrishnan *et al.*, “Retrofitting parallelism onto OCaml,” *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408995.
- [31] T. Petricek, D. Orchard, and A. Mycroft, “Coeffects: a calculus of context-dependent computation,” in *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, in ICFP '14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: 10.1145/2628136.2628160.
- [32] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” *Electronic Notes in Theoretical Computer Science*, vol. 203, no. 5, pp. 263–284, 2008, doi: 10.1016/j.entcs.2008.05.029.
- [33] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A Core Quantitative Coeffect Calculus,” in *Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410*, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: 10.1007/978-3-642-54833-8_19.
- [34] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for differential privacy,” *SIGPLAN Not.*, vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: 10.1145/1932681.1863568.
- [35] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, in POPL '99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: 10.1145/292540.292555.
- [36] D. E. Denning, “A lattice model of secure information flow,” *Commun. ACM*, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.
- [37] U. Dal Lago and F. Gavazzo, “A relational theory of effects and coeffects,” *Proc. ACM Program. Lang.*, vol. 6, no. POPL, Jan. 2022, doi: 10.1145/3498692.
- [38] R. P. James and A. Sabry, “Yield: Mainstream Delimited Continuations,” in *First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011)*, 2011, pp. 20–32. [Online]. Available: <https://homes.luddy.indiana.edu/sabry/files/yield.pdf>